PROF. DR.-ING. MARK SCHUTERA

FULL STACK HANDWERK

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.2

“SEINE WORT’ UND WERKE MERKT ICH UND DEN BRAUCH,

UND MIT GEISTESSTÄRKE TU ICH WUNDER AUCH.”

DER ZAUBERLEHRLING

Contents

Development and Production 9

Version Everything: Git & GitHub 21

Index 41

Introduction

This is an adaptation of the MIT Missing Semester course material. Well, by now it is that much adapted that it is basically an entirely novel thing, but the spirit and the motivation are the same. And oh boy, we need it here in Germany as well! In the Länd of the "Ingenieure", "Handwerk", and "Lehrlings" - we forgot the importance of practice and mastering our tooling skills.

Original course URL https://missing.csail.mit.edu/

As a mechanical engineer by training, I have always had this nagging feeling that something important was missing from my education. The more I transitioned into software engineering roles, the more this feeling intensified. Looking at your curriculum, there is no shortage of advanced concepts and theories to learn. This one fills the gap that many computer science programs overlook: practical proficiency with essential tools and workflows that every software engineer relies on. And truth be told, Professors rely on students having this basic proficiency already acquired, or to figure this stuff out on their own - which is fair in academia. For me, software engineering is also a practical discipline, and ought to be taught as such. I need you to know this stuff, so it is also fair to teach it.

This course will teach you everything you need to know to become proficient with the command line, version control systems like Git, text editors, shell scripting, and other essential tools that are the backbone of modern software development - in the end you will be able to develop and deploy your software solutions. This will make you a better software engineer, and it will make your life easier latest when you need to go full ML Lifecycle.

Development and Production

It's like Matrix, but for software.

The Why

Most software engineering curricula focus on algorithms, data structures, and the theory of computation. Writing a Python script that runs once on a single laptop is a skill; making that script available as a service, keeping it running, and recovering when it fails is a different one. This course addresses the second set of skills, the tools and workflows that sit between working code and the full stack of setting up and maintaining a running system.

Across ten lecture blocks we build PixelWise, a handwritten digit recognition web application. The starting point is two empty virtual machines; the end point is a deployed service including a frontend, and a backend with a trained model, an API, and a database, all running on Linux servers and connected over a network, with version control, automated deployment, persistent storage, and monitoring. Each tool we introduce is motivated by a specific need that arises in the construction of a fullstack application: Pixelwise.

PixelWise is the narrative thread of the entire course. Users draw a digit on a pixelated 28×28 canvas in the browser, the backend runs an image classifier trained on MNIST, the model predicts class and confidence, and results are stored in a database.

Development and Production.

This enables you to deploy your own prooducts and services in the future.

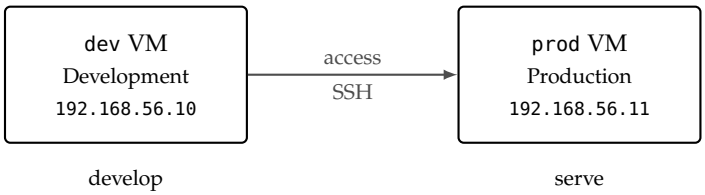

Figure 1: The two-machine setup. The dev VM is where code is written and tested; the prod VM is where the deployed application runs. Code travels from dev to prod through a deliberate deployment step over SSH.

The figure shows the starting arrangement: two virtual machines that play distinct roles throughout the course. The `dev` VM is the development environment in which code is written, modified, and tested. The `prod` VM is the production environment in which the

deployed application runs. Later blocks add the backend stack, the frontend, persistence, and automation between the two, but the separation between development and production stays intact from this block onward.

Hands On Experience

CONSIDER A SMALL WEB APPLICATION running on your laptop. A classmate two time zones away wants to try it; you send them the address. It works, until you close the lid to catch a train. That evening the Wi-Fi has changed, the address is different, and the process you started in the terminal is gone.

A SERVER is a machine whose job is to be available when no one is watching. It stays powered on, holds a stable address, and answers requests around the clock. Your laptop is where code is written and changed; the server is where code meets users. Keeping these two roles on separate machines is the first design decision of the course.

COUNT THE HOURS your laptop is actually reachable at a stable address. Subtract the time it is closed, asleep, on battery, behind a café router, or rebooting after an update. What remains is the service level a laptop alone can offer.

TWO MACHINES, ONE MENTAL MODEL. `dev` at 192.168.56.10 stands in for your laptop. `prod` at 192.168.56.11 stands in for the always-on host. Later lectures fill in the bridge.

THIS FIRST BLOCK focuses on machines and environments rather than on code. Before introducing a programming language, a version control system, or a framework, we need a place which is always up, to run programs, isolated from experiments, and a place for development. By the end of the block you will have provisioned both virtual machines,

- set up a server and a development machine as virtual machines on your laptop,
- learned the subset of Linux needed to navigate a server from the command line,
- and configured remote access over SSH using key-based authentication.

Virtual Machines

A VIRTUAL MACHINE IS A SIMULATED COMPUTER running inside your real computer. The software that creates and runs virtual machines is called a hypervisor. The hypervisor partitions the host's CPU, memory, and disk, and presents each virtual machine with what appears to be its own hardware. From the guest operating system's perspective, it is running on a physical machine.

HYPERVISORS are often categorised as type 1 or type 2. A type 1 hypervisor runs directly on the hardware, with the host operating system built into the hypervisor itself. Examples include VMWare ESXi and Microsoft Hyper-V in its server role; cloud providers use type 1 hypervisors to host the virtual machines you rent. A type 2 hypervisor runs as an application on top of a regular operating system. Oracle VirtualBox Manager is a type 2 hypervisor, which is why we can install it as a normal program on Windows, macOS, or Linux. The conceptual model is the same in both cases; the differences are in performance and deployment context.

ORACLE VIRTUALBOX MANAGER at `https://www.oracle.com/virtualization/virtualbox/` is the hypervisor we install on the host. Virtmanager and VMWare serve the same role with different trade-offs, and we could run this course on any of them.

ISOLATION is the primary reason to use a virtual machine. A misconfiguration, an uninstalled package, or a destructive command inside the guest does not affect the host operating system. Because the hypervisor abstracts away the hardware, the same VM runs identically on Windows, macOS, and Linux hosts, which is the basis of the reproducibility argument.

THERE ARE COSTS. A virtual machine executes a full operating system on top of the host, which carries a performance overhead that is typically small for I/O-bound work and larger for CPU-bound work. Each VM reserves some amount of RAM, disk space, and CPU time from the host, so the number of concurrent VMs is limited by available resources. Boot time is longer than launching an application but shorter than booting physical hardware. These costs are the price paid for isolation, reproducibility, and rollback.

A SNAPSHOT captures the entire machine state at a point in time, including disk contents, memory, and running processes. Snapshots are inexpensive to create and well-suited to exploratory work; making one before a risky change and restoring it afterwards is a standard pattern.

HOST-ONLY NETWORKING puts `dev` and `prod` on a private subnet, typically 192.168.56.0/24, that only the host and the guests can reach. The guests get internet through a second, NAT-mode adapter, so they can install packages without being exposed to the outside world. We assign static IPs, 192.168.56.10 for `dev` and 192.168.56.11 for `prod`, so that every command in the course refers to the same addresses.

OUTSIDE THE CLASSROOM, a production host is usually a rented or purchased server in a data centre, and the development machine is the laptop or workstation in front of you. Running both as VMs inside a single laptop is a teaching shortcut: it preserves the two-machine separation without asking you to rent hardware, and everything we learn here transfers directly when `prod` is later replaced by a cloud instance reached over the public internet.

Linux on the Server

LINUX is a family of Unix-like operating systems built around a common kernel, first released by Linus Torvalds in 1991. A distribution combines the kernel with a package manager, system libraries, and user-space tools into a coherent product. Ubuntu, Debian, Fedora, and Arch are distributions that differ in release cadence and defaults but share most of what matters at the command line.

A SERVER IN THIS SECTION is a computer configured to run services and accept connections over a network, typically without a graphical session. You interact with it through a shell, a program that reads commands and prints their output; the shell we use is Bash, the default on Ubuntu.

FIVE COMMANDS are sufficient for the first session. `ls` lists a directory, with `-la` showing permissions, ownership, and hidden files. `cd` changes directory; `cd ..` moves up one level and `cd ~` returns home. `pwd` prints the current path, `mkdir` creates a directory, and `cat` prints a file to the terminal.

UBUNTU at `https://ubuntu.com/` is the distribution we use on both VMs. Long-term support releases receive five years of security updates, which is why we pick the LTS ISO rather than the latest release.

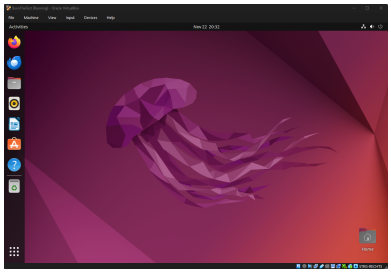

Figure 2: A running Ubuntu guest inside the Oracle VirtualBox Manager. The guest believes it owns a physical machine; the hypervisor is what makes that illusion hold.

LINUX IS A MULTI-USER OPERATING SYSTEM. Every file belongs to a user and a group and carries three permission sets, for owner, group, and everyone else. Each set independently grants read, write, or execute access. `whoami` reports the current user, and `ls -la` shows ownership and permissions:

```
-rw-r--r-- 1 user user 1234 Jan 15 10:00 config.txt
drwxr-xr-x 2 user user 4096 Jan 15 10:00 mydir/
```

THE UNIX PHILOSOPHY shapes the design of these tools. Each program does one thing well, programs pass streams of text to each other, and configuration lives in plain files. The result is a small vocabulary of composable commands that combine into pipelines no single command was designed for.

TWO COMMANDS modify these. `chmod` changes permissions and `chown` changes ownership:

```
chmod 600 ~/.ssh/id_ed25519          # owner-only read and write
sudo chown user:user file.txt        # assign user and group
```

READING THE MODE STRING from left to right, the first character tells you whether it is a file or directory, then three groups of `rwx` for owner, group, others. A dash means the permission is not granted. The leading `d` marks a directory.

The principle of least privilege applies throughout the course: grant each user, process, or service only the access it needs, so that narrow defaults limit the blast radius of a compromise or a mistake.

SSH, the Only Door

SSH, THE SECURE SHELL, is a protocol for operating a remote machine from a terminal over an encrypted channel. In the setup used in this course, SSH is the only means of access to the `prod` VM: there is no graphical login and no console attached. Misconfiguring SSH can lock a user out of the machine, which is why key material and the daemon's configuration are treated carefully in the exercises.

THE BASIC COMMAND takes the form `ssh <username>@<host>`; for example, `ssh produser@192.168.56.11` opens an encrypted connection to the server as the `produser` account. All input and output travel through this channel until the session is closed with `exit` or `Ctrl+D`.

PASSWORD-BASED AUTHENTICATION is vulnerable to guessing, brute-force attacks, and phishing. SSH supports an alternative in which the user presents a cryptographic key pair. The private key is kept on the client and never transmitted; the public key is placed on the server. During authentication the server issues a challenge that only the holder of the private key can answer, proving identity without the key itself ever leaving the client.

GENERATE AN ED25519 KEY PAIR on `dev`:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

The command writes the private key `id_ed25519` and the public key `id_ed25519.pub` to `~/.ssh/`. The private key must have permissions `600`, owner-only read and write, or SSH refuses to use it.

COPY THE PUBLIC KEY TO THE SERVER:

```
ssh-copy-id produser@192.168.56.11
```

This appends the key to `~/.ssh/authorized_keys`. Subsequent connections authenticate using the key pair and do not prompt for a password.

ONCE KEY AUTHENTICATION IS VERIFIED, password authentication can be disabled on the server. Edit `/etc/ssh/sshd_config` to set `PasswordAuthentication no`, then restart the daemon:

```
sudo systemctl restart ssh
```

OPENSSH at `https://www.openssh.com/manual.html` is the implementation shipped with Ubuntu and most other Linux distributions. The client on `dev` and the daemon on `prod` are both part of this project.

PUTTY at `https://www.putty.org/` is a legacy SSH client for Windows, from the years before OpenSSH shipped with the operating system. Windows 10 and later include `ssh.exe` natively, so PuTTY is rarely necessary; you may still encounter it in older tutorials and on locked-down corporate machines.

VS CODE REMOTE-SSH at `https://code.visualstudio.com/docs/remote/ssh` runs the editor locally while the language server, terminal, and file system live on the remote host. It reads `~/.ssh/config` and reuses the same keys as the `ssh` command, so any host you can reach with `ssh user@host` is one click away inside the editor.

ED25519 keys use elliptic-curve cryptography and produce shorter keys than the older RSA algorithm while providing comparable or stronger security. They are the default choice on modern SSH clients.

MANAGING PRIVATE KEYS. Private keys live in `~/.ssh/` on the machine that initiates connections, protected by filesystem permissions and, when generated with a passphrase, by symmetric encryption. `ssh-agent` holds the decrypted key in memory for the session so the passphrase is typed once per login. Password managers such as KeePass at `https://keepass.info/` or 1Password can store the passphrase and, with their SSH-agent integrations, supply the key to `ssh` without writing it to disk in plaintext. Keys are never synced to cloud storage in the clear, and a separate key pair per machine keeps the blast radius of a compromised laptop local.

After this change, password-based login attempts are rejected before credentials are even evaluated, which eliminates brute-force and credential-stuffing as viable attacks against the machine. The exercise introduces `nano` as a terminal editor and `systemctl` as the interface to the `systemd` service manager, both of which return throughout the course.

Exercise: Provisioning Virtual Machines

INSTALL THE ORACLE VIRTUALBOX MANAGER for your host OS with default settings, and download two Ubuntu LTS ISOs: the Desktop edition for `dev` and the Server edition for `prod`. The asymmetry is deliberate, `dev` carries a graphical environment so you can run an editor and a browser locally, while `prod` stays headless because a production server has no business running a desktop. You can also decide to run dev on your machine directly, but this is not further supported in the excercises.

EXERCISES are for practice and reinforcing concepts. Work through them yourself first, break things, discuss, this is not a time trial. If your VM ends up in a state you cannot recover, restore the `B1-fresh` snapshot and start over. At the end of the session, take a snapshot named `B1-complete`, your safety net for the next block.

CREATE THE `dev` VM in the Oracle VirtualBox Manager. Click `New` and configure:

- Name `dev`, Type `Linux`, Version `Ubuntu (64-bit)`
- Memory at least 4096 MB, CPUs at least 2
- Hard disk at least 25 GB, VDI, dynamically allocated

WHERE TO GET THEM. VirtualBox: `https://www.oracle.com/virtualization/virtualbox/`. Ubuntu Desktop LTS ISO: `https://ubuntu.com/download/desktop`. Ubuntu Server LTS ISO: `https://ubuntu.com/download/server`. Pick the LTS release in both cases, not the interim one, so support extends across the semester.

CREATE THE `prod` VM with the same procedure but tighter resources, since it runs headless:

- Name `prod`, Type `Linux`, Version `Ubuntu (64-bit)`
- Memory at least 2048 MB, CPUs at least 2
- Hard disk at least 10 GB, VDI, dynamically allocated

INSTALLER CHOICES that matter: use distinct credentials per VM, `user/password` on `dev` and `produser/prodpassword` on `prod`. The asymmetry is deliberate, you should feel that logging into `prod` is a different act from working on `dev`.

ATTACH THE MATCHING ISO to each VM. VirtualBox offers an unattended installation checkbox during creation, don't select it, you want the interactive installer. Once a VM exists, select it in the manager and open `Settings`, `Storage`. Under `Storage Devices` you will see a controller, `IDE` or `SATA`, with an `Empty` optical drive beneath it. Click that `Empty` entry, then on the right under `Attributes` click the small disc icon next to `Optical Drive`, choose `Choose a disk file`, and pick the Desktop ISO for `dev` or the Server ISO for `prod`. Confirm with `OK`, start the VM, follow the installer, when in doubt go with the defaults and just forward-enter.

WHY THE SPREAD. Ubuntu Desktop with GNOME wants roughly 4 GB of RAM and 25 GB of disk to feel responsive. Ubuntu Server has no GUI and runs comfortably in a quarter of that. Sized this way the pair fits inside a 6 GB / 35 GB envelope on the host.

AFTER THE FIRST BOOT OF EACH VM (BOTH DEV AND PROD), INSTALL THE OPENSSH SERVER TO ENABLE REMOTE ACCESS:

```
sudo apt update
sudo apt install openssh-server
```

This step is required for both Desktop and Server editions. Reboot after installation.

HOST-ONLY NETWORK. In `File`, `Host Network Manager` (or `File`, `Tools`, `Network Manager` on version 7+), create a host-only network if none exists, typically 192.168.56.1/24. **Disable DHCP** for this network so you can assign static IPs manually.

Configure the network adapters for each VM: Select the VM in VirtualBox Manager and open `Settings`, then go to the `Network` section. For Adapter 1, check "Enable Network Adapter" and set it to `NAT` (for internet access). For Adapter 2, select the Adapter 2 tab, check "Enable Network Adapter," set it to `Host-only Adapter`, and choose the host-only network you created.

Missing Adapter 2? Power-off the VM, make sure you are in Expert-mode, go to `Settings`, `Network`, select Adapter 2, check "Enable Network Adapter," and set it to `Host-only Adapter`.

Assign static IPs inside each VM:

1. Boot the VM and log in.
2. Run `ip a` to list network interfaces. The host-only adapter is usually `enp0s8`, but confirm whether the name shows up in the output.
3. Edit the Netplan config with `sudo nano /etc/netplan/00-installer-config.yaml` Save and exit nano with (Ctrl+X, Y, Enter).
4. For `dev`, set the config as:

```
network:
  version: 2
  ethernets:
    enp0s8:
      addresses:
        - 192.168.56.10/24
```

5. Apply the config with `sudo netplan apply`
6. Check the new address with `ip a` again; you should see the static IP assigned to `enp0s8`.

Open a terminal (Desktop). On Ubuntu Desktop press `Ctrl+Alt+T`, or open `Activities` (top-left) and type "terminal"; press Enter. You can also right-click the desktop and select "Open Terminal" if the menu is present.

For `prod`, use `192.168.56.11/24` instead.

Netplan permission warning. Netplan will warn if a configuration file is writable by group/others or not owned by `root`. This is a security check to avoid loading untrusted configuration files; expected ownership is `root:root` and modes such as `644` for the YAML files.

Verify connectivity: On `dev`, run `ping 192.168.56.11` to test if `prod` responds. If you get no reply, double-check the Netplan configuration (correct interface and address), ensure Adapter 2 is set to Host-only in both VMs, and confirm that both VMs are running and attached to the same host-only network.

Troubleshooting. If the interface name is not `enp0s8`, use the correct one from `ip a`.

```
user@dev:~$ ping 192.168.56.11
PING 192.168.56.11 (192.168.56.11) 56(84) bytes of data.
64 bytes from 192.168.56.11: icmp_seq=1 ttl=64 time=1.63 ms
```

Figure 3: ping

Exercise: Navigating Linux

THIS EXERCISE turns the five commands and the permission model into something your hands remember. You walk the directories common to every Linux system, identify yourself to the machine, and read and change a file's mode string. By the end, the output of `ls -la` should read as a description you can speak aloud rather than a blob of symbols.

TEN-FINGER TYPING is part of the toolchain once the terminal is your main tool. A server has no menus, so your throughput is bounded by how quickly you can speak to the shell without looking at the keys; every hunt for a slash, tilde, or brace is attention spent on the keyboard instead of the problem. Ten minutes a day at `https://www.keybr.com/`, `https://typing.io/`, or `https://monkeytype.com/` is enough to remove that tax over a few weeks.

EXPLORE THE FILESYSTEM by visiting `/home`, `/etc`, and `/opt` in turn with `cd` and `ls -la`, and read two files with `cat`:

```
cat /etc/hostname
cat /etc/os-release
```

THE FILESYSTEM AS MAP. `/home` is where users live, `/etc` holds configuration, and `/opt` is where PixelWise will be installed in a later block. These locations reflect decades of convention you will meet on every Linux machine; knowing them lets you locate things by instinct.

CREATE A WORKSPACE and check that you are where you think you are:

```
mkdir ~/workshop && cd ~/workshop
pwd
touch notes.txt
rm notes.txt
```

`pwd` is a reassurance command. When something does not behave as you expect, asking the shell where it thinks it is usually explains the problem faster than re-reading the error.

UNDERSTAND WHO YOU ARE and look at the permissions on your new file:

```
whoami
id
ls -la notes.txt
```

Decode the first column of `ls -la` against the margin note on mode strings from Block 2. Name aloud what each of the ten characters means; if you can do that without looking, permissions have stopped being mysterious.

CHANGE THE MODE AND WATCH IT:

```
chmod 600 notes.txt
ls -la notes.txt
```

The next exercise will require `600` on your SSH private key, or OpenSSH refuses to use it; you have now seen what `600` looks like and why ownership matters.

A THOUGHT EXPERIMENT ON LEAST PRIVILEGE. Your `user` account can read `/etc/os-release` but cannot edit `/etc/ssh/sshd_config` without `sudo`. Why does that separation exist, and what would change if the web service we deploy in a later block ran as `root`? Name one file an attacker who compromised such a service could reach that a non-root service could not; the mental habit matters more than the exact file.

Exercise: SSH and Key-Based Access

THIS EXERCISE walks through three states of remote access in order: password login, key-based login, and password login disabled. The point is not the commands but to recognise which state you are in at any moment and why each is stronger than the last.

FIRST, LOG IN WITH A PASSWORD. From `dev`, open a session to `prod`:

```
ssh produser@192.168.56.11
```

Type your password, look around briefly, and log out with `exit`. This is the baseline. Every later step is interpretable as an upgrade over this state.

GENERATE A KEY PAIR on `dev` and install it on `prod`:

```
ssh-keygen -t ed25519 -C "you@example.com"
ssh-copy-id produser@192.168.56.11
```

Reconnect with `ssh produser@192.168.56.11` and notice the password prompt is gone. Log in and inspect what `ssh-copy-id` wrote for you:

```
cat ~/.ssh/authorized_keys
```

WHAT JUST HAPPENED. `ssh-copy-id` appended the contents of `id_ed25519.pub` on `dev` to `~/.ssh/authorized_keys` on `prod`. At the next connection the daemon issued a challenge that only the private key on `dev` could answer; no secret material ever left the client. Seeing your public key in a readable file anchors the whole exchange to something concrete.

DISABLE PASSWORD AUTHENTICATION. Edit `/etc/ssh/sshd_config` with `sudo nano`, set `PasswordAuthentication no`, and restart the daemon:

```
sudo systemctl restart ssh
sudo systemctl status ssh
```

The `status` output should show `active (running)`. If it does not, keep your current session open and fix the configuration before logging out; a locked-out VM is rescued by restoring the snapshot, not by wishful thinking. From a machine without the key, the daemon now rejects the attempt without even asking for a password.

TAKE A SNAPSHOT of both VMs once everything works, naming them `B1`; this is your known good state for the rest of the course. Then deliberately break something inside a VM, watch the fallout, and restore the snapshot. You now have physical evidence that the safety net holds.

NUKE THE MACHINE. From `prod`, run `sudo rm -rf /` to delete all files on the machine. Restore from the snapshot.

RUN PROD WITHOUT GUI. In future run prod headless without GUI from the VirtualBox Manager. You can still log in with SSH and run commands, but there is no desktop to interact with. This is how a real production server works.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during and after the exercises:

- Why do we use two separate machines instead of running everything on one?
- Why is disabling password authentication a stronger protection than picking a longer password?
- What does the principle of least privilege mean in the context of file ownership, and where did it show up in today's workshop?
- If your laptop died tomorrow, how much of today's setup could you reproduce, and what would you lose?
- Which of the five Linux commands do you still have to look up, and which one surprised you?
- How does the two-machine model map to a real cloud deployment with staging and production? How is our development machine different from staging?

RECAP of Key Concepts:

- Virtual machines isolate experiments from the host and make snapshots a first-class safety net.
- The two-machine architecture separates development from production at the hardware level.
- Files on Linux carry owners, groups, and three sets of permissions, and `chmod` and `chown` manage both.
- SSH is the only door into the server, and key-based authentication replaces passwords with cryptography that cannot be guessed.

SO FAR, WE OWNED THE BOX. The next step is to own the history of every change we make to it. In the next chapter we put code and configuration under version control with Git and GitHub, turn today's manual setup into a script, and make what we just did something a teammate, or a fresh VM, can replay from scratch.

A NOTE ON WHAT COMES NEXT. The commands you ran in this session are reproducible but not yet reproduced. You ran them in a terminal, they worked, and they are gone. In the next block we place them under version control and turn the sequence into a script that any future VM can run end to end, without you remembering anything.

WITHOUT SHIPPING ANYTHING YET, we already have two machines that talk to each other, a secure door, and a known good state to fall back on.

TEASER. Every command we ran today was reproducible, but none of it was reproduced. How do we capture the steps so that any fresh VM can be brought to the same state without human memory?

Version Everything: Git & GitHub

Git a life.

The Why

Software does not stop changing. Requirements shift, collaborators join, bugs appear, and experiments turn into features that later have to be unwound. Without a deliberate record of what changed, when, and why, the past becomes guesswork and rollback becomes impossible. This chapter introduces the tool every working codebase uses to keep the past reachable and the future safe to explore.

Across the ten lecture blocks we build PixelWise, the full stack of tools we introduce sits on top of version control from block 2 onward. Every configuration file, every script, and every line of backend code is committed, pushed, and recoverable. The server setup we ran manually in block 1 becomes a script that any fresh VM can replay end to end. What was ephemeral knowledge in a single terminal turns into durable, auditable state that moves between machines without anybody having to remember what they typed.

PixelWise continues to be the narrative thread. Block 1 left two empty virtual machines on your laptop; by the end of this block, one of them hosts a `pixelwise` repository tracked by Git, published on GitHub, and cloned onto the server at `/opt/pixelwise`.

Hands On Experience

You have been there. A folder full of copies: `report.py`, `report_v2.py`, `report_final.py`, `report_FINAL_fixed.py`. Which one is current? Which one works? What changed between them? Nobody knows, including you, two weeks from now.

Version control solves this permanently. Every change is recorded, annotated, and reversible; you can see exactly what changed, when, by whom, and why; you can return to any previous state; and you can work on multiple ideas in parallel without

Count the time you have lost to guessing which file was current, reconstructing a change you made yesterday, or restoring a working version from a zip you made "just in case". Version control removes the class of problem, not a single instance of it.

them interfering with each other. Git is the version control system we adopt, and it is the de facto standard for the rest of the industry.

THIS SECOND BLOCK focuses on Git and GitHub rather than on application code. Before we write a line of backend logic, every artefact we produce from here on must be tracked, reviewable, and reproducible on a new machine. By the end of the block you will have

- initialised a local Git repository for PixelWise on the `dev` VM,
- captured the manual server setup from block 1 in a reproducible shell script,
- published the repository on GitHub over SSH, and
- cloned the repository onto the `prod` VM, so both machines share a single source of truth.

The Git Data Model

WORRY NOT, git gets easier once you get the basic idea that branches are homeomorphic endofunctors mapping submanifolds of a Hilbert space, or so someone on Reddit said. In practice the data model is a lot more approachable than the joke suggests, and the rest of this section spells it out.

GIT IS NOT A BACKUP TOOL. It is a content-addressable filesystem with a version history layered on top, and understanding that model makes every command intuitive rather than a sequence of incantations. Under the cover of `git add` and `git commit` there are four kinds of objects and two pointers that together express every state a repository has ever been in.

Blob A file's contents, stored and addressed by the SHA-1 hash of those contents. The name is irrelevant to Git; only the bytes matter.

Tree A directory listing that maps filenames to blobs for files and to other trees for subdirectories.

Commit A snapshot of the entire project at a point in time, combining a pointer to the root tree, the author, a timestamp, a message, and a pointer to the parent commit or commits.

Branch A lightweight, movable pointer to a commit. `main` is a branch. Creating one is instantaneous because it only creates a new pointer.

HEAD A pointer to the branch you are currently on; when you commit, `HEAD` moves forward along with the branch it points to.

EVERY COMMIT IS IDENTIFIED BY A SHA-1 HASH, a 40-character hexadecimal string such as `a3f2b7c...` computed from the commit's contents. If anything changes, a single character in a single file, the hash changes, which is what makes the history tamper-proof and why two repositories with the same hashes are known to contain the same objects.

GIT at `https://git-scm.com/` was created by Linus Torvalds in 2005 to manage the Linux kernel source code after the previous tool's licence terms changed. It is now the de facto standard for version control across the industry.

Pro Git book, free: `https://git-scm.com/book/en/v2`.

CONTENT-ADDRESSABLE STORAGE is the idea that an object's identifier is a function of its bytes. Git inherits this pattern from earlier systems such as Monotone and the Plan 9 Venti archival store; Docker image layers and IPFS use the same trick. If you know the identifier, you already know whether you have the right object.

The Staging Workflow

GIT HAS THREE AREAS in which your changes live before they become part of the permanent history.

Working directory The files on disk, what your editor reads and writes.

Staging area Sometimes called the index, the set of changes you have marked for the next commit. A deliberate selection, not a side effect of editing.

Repository The committed history: permanent, immutable snapshots linked by parent pointers.

A HANDFUL OF COMMANDS move changes between those three areas.

```
git status                # what changed? what is staged?
git add file.py           # stage a file for the next commit
git add .                 # stage everything, use with care
git commit -m "message"   # commit the staged changes
git diff                  # unstaged changes vs last commit
git diff --staged         # staged changes vs last commit
git log                   # show commit history
git log --oneline         # compact view, one line per commit
```

THE SEPARATION BETWEEN STAGING AND COMMITTING is what gives Git its fine-grained control. A single working session might touch five files, and you can group them into two or three commits that each tell a coherent story, rather than one commit that mixes unrelated changes. In practice this means the staging area is where you compose the history you want to keep, not just a technical step on the way to `git commit`.

WRITE COMMIT MESSAGES IN THE IMPERATIVE MOOD: "Add classifier module" rather than "Added classifier module." The message completes the sentence "This commit will . . . ". Keep the first line under 72 characters and leave a blank line before any longer explanation.

The `.gitignore` File

NOT EVERYTHING BELONGS IN VERSION CONTROL. Compiled byte code is regenerated on every run, virtual environments are rebuilt from a lockfile, trained model artefacts are larger than a repository should carry, and secrets must never be tracked at all. The `.gitignore` file tells Git which paths to exclude from tracking:

```
# Python
__pycache__/
*.pyc
.venv/

# Secrets
.env

# Data and models, too large and reproducible
data/
models/*.pkl

# Editor files
.vscode/
*.swp
```

TREAT A COMMITTED SECRET AS COMPROMISED. If a password, API key, or private key has entered a commit, assume somebody now holds it and rotate it immediately. Rewriting history with `git filter-branch` or BFG Repo-Cleaner at https://rtyley.github.io/bfg-repo-cleaner/ removes the secret from future clones, but it does not recall any clone that already exists. The principle of least privilege from block 1 has its version control counterpart here: the narrower the set of files Git tracks, the smaller the blast radius of a mistake.

GIT HISTORY IS PERMANENT. A secret committed once is leaked forever, even if you delete the file in a later commit: the old commit still contains it, and any clone carries the full history. `.gitignore` is the first line of defence, and it only works when the entry is in place before the first commit that would otherwise include the file.

Branching and Merging

BRANCHES LET YOU WORK ON IDEAS IN ISOLATION. The `main` branch is the stable baseline that always builds and always runs. A feature branch diverges from `main` at a specific commit, evolves through any number of commits of its own, and merges back once the work is ready.

```
git branch                    # list branches
git branch feature-x          # create a new branch
git checkout feature-x        # switch to it
git checkout -b feature-y     # create and switch in one step
```

WHEN THE FEATURE IS READY, you merge it back into the baseline:

```
git checkout main
git merge feature-x
```

A MERGE CONFLICT occurs when two branches changed the same line of the same file and Git cannot decide which version to keep. Rather than guessing, it marks the conflict in the file and asks you to resolve it:

```
<<<<<<< HEAD
result = model.predict(x)
=======
result = classifier.classify(x)
>>>>>>> feature-x
```

Edit the file to keep the correct version, remove the markers, then stage and commit the resolution:

```
git add file.py
git commit -m "Resolve merge conflict in file.py"
```

A CONFLICT IS NOT A FAILURE. It is Git refusing to invent an answer you did not give it. Treat conflicts as a prompt to re-read both sides, understand which change should survive, and write the resolution as a small, explicit commit of its own.

Tags

A TAG MARKS A POINT IN HISTORY THAT MATTERS, a release, a milestone, or a known-good state you may need to return to. Unlike a branch, a tag does not move once it is created.

```
git tag v0.1                            # lightweight tag
git tag -a v0.1 -m "initial setup"      # annotated tag, preferred
git push --tags                         # push tags to remote
```

SEMANTIC VERSIONING is the convention we follow: versions take the form `MAJOR.MINOR.PATCH`, and each component carries meaning about the kind of change it represents. A `v0.x` tag signals that the project is still pre-release and that the public interface can still change without warning, which is exactly where PixelWise sits today.

LIGHTWEIGHT VS ANNOTATED TAGS. A lightweight tag is only a name pointing to a commit. An annotated tag stores the tagger, the date, and a message, and it is what releases should use. We tag `v0.1` in this block; the tag becomes meaningful when you look back at it in block 8, after seven more iterations have rolled past it.

Remotes and GitHub

A REMOTE IS A COPY OF YOUR REPOSITORY ON ANOTHER MACHINE, usually a hosting service such as GitHub. Remotes are how a repository crosses machines, how collaborators share work, and how code travels from `dev` to `prod` later in the course.

```
git remote add origin git@github.com:user/pixelwise.git
git push -u origin main        # push and set upstream
git pull                       # fetch plus merge from remote
git fetch                      # fetch without merging
git clone <url>                # copy a remote repo locally
```

A PULL REQUEST is a proposal to merge a branch into another branch, typically a feature branch into `main`. It is the standard mechanism for code review: changes are visible, reviewable, and discussable before they become part of the main codebase. A pull request is a GitHub feature, not a Git one; at the Git level, the merge itself is what happens once the review passes.

GITHUB IS NOT GIT. Git is the version control system; GitHub is a hosting platform that adds pull requests, issues, CI/CD, and collaboration features on top of Git.
GitHub: https://github.com/
GitHub Docs: https://docs.github.com/

SSH AGAIN. The remote URL `git@github.com:user/pixelwise.git` uses the same SSH protocol you configured in block 1. GitHub holds your public key; your private key stays on `dev`. The key-based authentication that replaced passwords on `prod` replaces them here as well, and the habit of adding your public key to each new service generalises to every remote system the rest of the course touches.

Beyond GitHub

GitHub is not the only option. Git is decentralised by design, and any machine that can accept SSH connections can host a repository. Picking a platform is a trade-off between convenience, cost, and sovereignty over your data. A short tour of the landscape keeps the choice explicit rather than accidental.

GitLab The full-featured GitHub alternative. Strong built-in CI/CD, issue boards, and release tooling. Runs as a hosted service at `gitlab.com` or as GitLab Community Edition on your own server. Choose it if you want one integrated DevOps stack.

Bitbucket Part of the Atlassian ecosystem. Integrates tightly with Jira and Confluence, a natural fit for teams that already live in Atlassian tools.

Gitea A lightweight, self-hosted forge written in Go. Runs on a Raspberry Pi, installs in minutes, and looks and feels like a stripped-down GitHub. Choose it when you want simple self-hosting without the resource weight of GitLab.

Forgejo A community-driven soft fork of Gitea. Looks, acts, and installs almost exactly like Gitea, but is governed as a non-profit community project with no for-profit company behind it. Choose it if you want the Gitea experience with a free-software purist stance.

Bare SSH repository The minimal option. A directory created with `git init -bare` on any SSH-reachable machine is enough to collaborate: `git clone user@host:/path/to/repo.git`. No web UI, no pull requests, just Git.

The choice is reversible. Because the Git data model is independent of any platform, a repository can be pushed to GitHub today, mirrored to GitLab tomorrow, and archived on a bare SSH host the year after. We use GitHub in this course for familiarity, but nothing we learn here is specific to it.

Where to look. Self-hosted and alternative platforms:
GitLab: `https://about.gitlab.com/`
Gitea: `https://about.gitea.com/`
Forgejo: `https://forgejo.org/`
Bitbucket: `https://bitbucket.org/`
Bare repo guide:
`https://git-scm.com/book/en/v2/Git-on-the-Server-Getting-Git-on-a-Server`

The Setup Script

On the server VM, you installed packages manually in block 1 by typing commands into a shell. Those commands worked, and then they were gone. A fresh VM, a new teammate, or a rebuild after a failure would need the same sequence run again, and remembering which commands you ran last week is not a plan. A short shell script captures the work once and reruns it on demand.

```
#!/bin/bash
# setup-server.sh -- provision a fresh server VM
set -euo pipefail

sudo apt update
sudo apt install -y git python3 python3-pip python3-venv curl
```

This is the first file that justifies the whole chapter. It is small, it is boring, and it turns a bundle of remembered terminal commands into an artefact you can commit, tag, and reproduce on any new machine. Every later block adds to it: a Python environment in block 3, a systemd unit in block 5, a reverse proxy in block 7. By block 8 the script provisions an entire service stack from nothing.

The `set -euo pipefail` line makes the script fail immediately on any error with `-e`, on any undefined variable with `-u`, and on pipe failures with `-o pipefail`. Without it, a failing `apt install` prints a message and the script continues anyway, which is how servers end up in half-configured states that nobody can debug later.

Exercise: Initialising the Repository

BLOCK 2 INTRODUCED SEVEN IDEAS in the theory section: the Git data model, the staging workflow, `.gitignore`, branching and merging, tags, remotes and GitHub, and the setup script. Each exercise below mirrors one of them in the same order, so you can read a section and immediately practise what it described.

EXERCISES are for practice and reinforcing concepts. Work through them yourself first, break things, discuss; this is not a time trial. If your VM ends up in a state you cannot recover, restore the `B1-complete` snapshot from block 1 and start over. At the end of the session take a snapshot named `B2-complete`; it is the starting state for block 3.

CONFIGURE YOUR GIT IDENTITY on `dev` so every commit carries your name and email address:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

INSTALL GIT FIRST. On both VMs run `sudo apt update && sudo apt install -y git` so the commands below have something to execute. You will capture this install step properly in the final exercise, together with the rest of the server's packages.

CREATE THE PROJECT DIRECTORY and initialise a repository:

```
mkdir ~/pixelwise && cd ~/pixelwise
git init
```

Run `git status`; Git should report that you are on branch `main` with no commits yet.

LOOK INSIDE `.git/`. `ls -la .git` shows the on-disk form of the data model from block 2: `objects/` for blobs and trees, `refs/` for branches and tags, `HEAD` as a plain file that names the current branch. Every Git command you run from here on manipulates exactly these files.

Exercise: Staging and Committing

Write a `README.md` with a one-line description of what PixelWise is. Stage it, inspect the status, commit, and read the resulting history:

```
git add README.md
git status
git commit -m "Add README with project description"
git log --oneline
```

Watch the three areas. Edit `README.md`, add a second line, and run `git diff` before staging and `git diff -staged` after:

```
git diff
git add README.md
git diff --staged
git commit -m "Expand README with a second line"
```

Naming the difference between the two diff outputs aloud is the fastest way to make the working directory, staging area, and repository stop feeling abstract.

Read the hash Git prints after the commit. That seven-character prefix is the start of the SHA-1 from block 2, and any unambiguous prefix of a commit's hash refers to that commit. Running `git log` later should always show this commit at the root of the history.

Exercise: Guarding Secrets

WRITE A `.gitignore` that covers the paths you already know should stay out of version control:

```
# Python
__pycache__/
*.pyc
.venv/

# Secrets
.env

# Data and models
data/
models/*.pkl

# Editor files
.vscode/
*.swp
```

TEST THAT IT WORKS. Create a dummy `.env` with a fake secret inside and run `git status`:

```
echo "API_KEY=not-a-real-secret" > .env
git status
```

Stage and commit `.gitignore` itself so the rule is part of the repository from the start:

```
git add .gitignore
git commit -m "Add gitignore for Python, secrets, data, and editors"
```

WHAT YOU EXPECT TO SEE. `.env` does not appear in the status output because `.gitignore` already excluded it. If it does appear, the `.gitignore` was written after the file was tracked, and `git rm -cached .env` is the fix. This is why `.gitignore` is the very first configuration file to commit in a new repository.

Exercise: A Feature Branch

Warm up in a sandbox first. Before touching the real repository, complete the introduction sequence on the `Main` tab at Learn Git Branching: https://learngitbranching.js.org/. The four interactive puzzles walk you through commits, branches, merges, and rebases with a live commit graph next to the command line, which is the most direct way to make the data model from block 2 concrete. Finish at least `Introduction to Git Commits`, `Branching in Git`, `Merging in Git`, and `Rebase Introduction` before you continue; each one takes a few minutes and resets with a click if you get stuck.

Two more sandboxes worth keeping open while you work:
Git Exercises: https://gitexercises.fracz.com/
Oh My Git!: https://ohmygit.org/
Git Exercises poses command-line puzzles against a real Git clone, while Oh My Git! is a card game that shows the graph changing as you play. Pick the format that matches how you learn, but do not skip this step; branching is the one Git topic that fluency in a sandbox transfers directly to.

Now create a branch on `pixelwise` to extend the README without touching `main`:

```
cd ~/pixelwise
git checkout -b improve-readme
```

Add a short section to `README.md` describing the two-machine setup from block 1, then commit:

```
git add README.md
git commit -m "Describe two-machine setup in README"
```

Merge back into `main`:

```
git checkout main
git merge improve-readme
git log --oneline --graph --all
```

Read the graph. `git log -oneline -graph -all` draws the commit graph as ASCII art. With one feature branch that merged cleanly the graph is a straight line, but it is the same visualisation you saw in the Learn Git Branching puzzles, and the same one you will use to untangle complicated merges later.

Provoke and resolve a conflict. Back in a browser, go to the `Remote` tab of Learn Git Branching and complete `Push & Pull - Git Remotes!` and `To Origin And Beyond - Advanced Git Remotes!`, then try the `Mixed Bag` puzzles that require rebasing a conflict. The sandbox shows every intermediate state of the graph, so when the same situation shows up on `pixelwise` you already know which commit is which and what the resolution should look like.

Delete the merged branch once the work is on `main`, so the branch list stays a view of active work rather than a graveyard:

```
git branch -d improve-readme
```

Exercise: Tagging `v0.1`

MARK THIS POINT IN HISTORY with an annotated tag:

```
git tag -a v0.1 -m "Initial repository with README and gitignore"
git log --oneline --decorate
```

The `-decorate` flag shows branches and tags alongside each commit, so `v0.1` is visible on the commit it points to.

INSPECT WHAT THE TAG HOLDS:

```
git show v0.1
```

The output names the tagger, the date, the annotation message, and the commit the tag points at. A lightweight tag would show only the commit; the extra metadata is exactly what makes annotated tags appropriate for releases.

PUSHING TAGS IS SEPARATE. A regular `git push` does not include tags by default. You will use `git push -tags` in the next exercise once the repository has a remote, which is deliberate: tags name releases, and releases are a conscious act.

Exercise: Publishing to GitHub

Create a GitHub account if you do not have one, and add your `dev` VM's SSH public key under `Settings, SSH and GPG keys`. The public key is the one in `~/.ssh/id_ed25519.pub` on `dev`; print it with `cat` and paste the contents into GitHub.

Same key, new service. You are reusing the SSH key generated in block 1 rather than creating a new one. One key per machine is enough; you add the public half to every service that should accept connections from `dev`. The private half never leaves the machine it was generated on.

Create an empty repository on GitHub named `pixelwise` with no README, no `.gitignore`, and no licence, since you already have all three locally. Then wire the local repository up and push:

```
git remote add origin git@github.com:your-user/pixelwise.git
git push -u origin main
git push --tags
```

Open a pull request to experience the review flow on your own repository. On GitHub, edit the README through the web interface on a new branch called `readme-tweak`, commit the change, and click `Compare & pull request`. Review the diff, merge the pull request, then pull the result back to `dev`:

```
git checkout main
git pull
```

What a pull request does. The pull request renders the diff, collects reviewer comments, and runs any configured checks before the merge button lights up. For a solo push this is ceremony; for a team repository it is where most of the engineering discussion happens, and the habit of opening one is worth forming while the stakes are still low.

Contribute a pull request to an open-source project before the next class. Pick any project on GitHub, GitLab, or a self-hosted forge and submit a pull request. It can be a typo fix in the README, a clarifying sentence in the documentation, a missing example, or a small bug fix. The goal is to practise the full collaboration loop end to end: fork, clone, branch, commit, push, open the PR, and respond to review comments. Projects you already use are the best starting point; otherwise browse curated lists of beginner-friendly issues:

```
https://goodfirstissue.dev/
https://github.com/MunGell/awesome-for-beginners
```

Bring the PR link to the next session, whether it has been merged or not.

Exercise: The Reproducible Setup Script

SSH INTO THE SERVER VM and install the tools the rest of the course needs:

```
sudo apt update
sudo apt install -y git python3 python3-pip \
    python3-venv curl
```

Verify the install with `git -version` and `python3 -version`.

BACK ON `dev`, capture exactly what you just ran in a script. Inside the `pixelwise` repository, create `scripts/setup-server.sh`:

```
#!/bin/bash
# setup-server.sh -- provision a fresh server VM
set -euo pipefail

sudo apt update
sudo apt install -y git python3 python3-pip python3-venv curl
```

MAKE IT EXECUTABLE, COMMIT, AND PUSH:

```
chmod +x scripts/setup-server.sh
git add scripts/setup-server.sh
git commit -m "Add reproducible server setup script"
git push
```

WHY A SCRIPT, NOT A NOTE. A note tells a human what to do; a script does it. As PixelWise grows the script grows with it, and the fact that it always ran to completion is the evidence that a fresh VM really does reach the state the rest of the course assumes.

CLONE THE REPOSITORY ONTO THE SERVER so both machines share the same source:

```
sudo mkdir -p /opt/pixelwise
sudo chown produser:produser /opt/pixelwise
git clone git@github.com:your-user/pixelwise.git /opt/pixelwise
```

The server now carries the repository at `/opt/pixelwise`, and any `git pull` from there brings in changes you push from `dev`.

TAKE A SNAPSHOT of both VMs and name them `B2-complete`. From here on, every new artefact lives inside the repository, and the snapshot is your fallback if something in block 3 goes sideways.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during and after the exercises:

- What is the difference between `git add` and `git commit`, and why is the separation useful rather than an extra step?
- Why does a SHA-1 hash change when any byte of a file changes, and what guarantee does that give you about the history?
- Why should you never commit `.env` files or API keys, and what happens to a secret that was committed and then deleted in a later commit?
- When would you use a branch instead of committing directly to `main`?
- What is the difference between a lightweight tag and an annotated tag, and which one belongs on a release?
- How does a pull request differ from `git merge`, and which one does which part of the job?
- When would you reach for GitLab, Gitea, Forgejo, or a bare SSH repository instead of GitHub, and what changes about the workflow if you do?
- Why do we capture the server setup in a script rather than running the same commands manually on the next fresh VM?

RECAP of Key Concepts:

- Git stores content as blobs, grouped into trees, pointed at by commits, with branches as movable pointers and `HEAD` as the current one.
- The staging area is where you compose the next commit, rather than a technical hurdle before it.
- `.gitignore` keeps byproducts and secrets out of the history, and Git history is permanent enough that rotation, not deletion, is the right response to a leak.
- Branches isolate parallel work, and merges, pull requests, and conflicts are the means of bringing that work back together.

A NOTE ON WHAT COMES NEXT. The repository is in place, but it only holds configuration so far. Block 3 adds a Python environment, a package layout, and the first PixelWise source file, all of which will be staged, committed, and pushed through the workflow you just practised.

- Tags name points in history, and annotated tags are the right tool for releases.
- Remotes put your repository on another machine, and GitHub, GitLab, Gitea, Forgejo, and bare SSH hosts are interchangeable options for where that machine lives.
- A setup script turns remembered manual work into a reproducible artefact, and it is the first load-bearing file in PixelWise.

So far, we own the history. The next step is to start filling it with code. In the following chapter we create a Python environment on `dev`, lay out the PixelWise package, and commit the first real module, so the repository starts doing work rather than just holding configuration.

Without shipping anything yet, PixelWise already has a single source of truth that lives on both `dev` and `prod`, and a reproducible path from an empty VM to a configured one.

Teaser. The repository holds configuration and a script, but no application code. How do we turn an empty Python interpreter on `dev` into a project with pinned dependencies, a package layout, and a first module that the repository can track end to end?

Index

.gitignore, 24
  security, 27
/etc, 17
/opt, 17

Bash, 12
Bitbucket, 24

chmod, 12
chown, 12

dev VM, 9
distribution, 12

Ed25519, 13
exercises, 15, 17, 18

Forgejo, 24

Git, 19
  blob, 21
  branch, 22
  branching, 22
  commit, 21
  data model, 21
  HEAD, 22
  history, 25
  merge conflict, 23
  merging, 22
  remotes, 23
  self-hosting, 24
  staging, 22
  tags, 23
  tree, 21
Gitea, 24
GitHub, 19, 23
GitLab, 24

hypervisor, 11

issues, 23

KeePass, 13

least privilege, 12
license, 2
Linux, 9, 12

MNIST, 9

nano, 14

OpenSSH, 13
Oracle VirtualBox Manager, 9

PixelWise, 9
pull request, 23, 24
PuTTY, 13

recap, 19
reflection, 19
Remote-SSH, 13

server, 9
setup-server.sh, 25
snapshot, 11
SSH, 9, 13
ssh-agent, 13
systemctl, 14
systemd, 14

touch typing, 17
type-1 hypervisor, 11
type-2 hypervisor, 11

Ubuntu, 12
Unix philosophy, 12

version control, 19, 21
Virtual Machines, 9, 11
VS Code, 13

whoami, 12